



BARBEARIA Luck

TRADIÇÃO · ESTILO · PRECISÃO

Guia de Desenvolvimento Local

Setup, fluxo dev, comandos úteis, troubleshooting

Branch: `claude/barbearia-luck-app-LzpRD`

1. Setup base (uma vez só)

Pré-requisitos

- **Node.js 20+** — `winget install OpenJS.NodeJS.LTS`
- **Docker Desktop** — `winget install Docker.DockerDesktop` (com virtualização habilitada na BIOS + WSL 2)
- **Git** — `winget install Git.Git`

Clonar e preparar

```
# Clone fora do OneDrive (importante!)
mkdir C:\projetos
cd C:\projetos
git clone https://github.com/Rodrigotxs/LuckApp.git
cd LuckApp
git checkout claudelbarbearia-luck-app-LzpRD

# Cria .env do backend a partir do exemplo
Copy-Item backend\.env.example backend\.env

# Sobe Postgres + Redis via Docker
docker compose up -d

# Backend: instala deps, aplica migrations, faz seed
cd backend
npm install
npx prisma generate
npx prisma migrate deploy
npx prisma db seed
cd ..

# Frontend: instala deps
cd frontend
npm install
cd ..
```

⚠ **Nunca clone dentro do OneDrive.** O sync corrompe `node_modules` e `.git`. Use `C:\projetos\` ou similar.

2. Rodar o app (dia a dia)

Você mantém **3 coisas rodando** simultaneamente:

#	O QUE	ONDE
1	Docker Desktop (baleia verde na barra)	Aplicativo
2	<code>cd backend && npm run start:dev</code>	PowerShell #1
3	<code>cd frontend && npm run dev</code>	PowerShell #2

Abra `http://localhost:3000`. Login demo: `demo@barbearialuck.com` / `123456`.

Endpoints úteis quando estão rodando

URL	PARA QUÊ
<code>http://localhost:3000</code>	App (frontend)
<code>http://localhost:3001/api/docs</code>	Swagger — testar endpoints do backend
<code>http://localhost:5555</code>	Prisma Studio (rode <code>npx prisma studio</code> num 3º terminal)

3. Fluxo de alteração — o que fazer a cada tipo de mudança

VOCÊ MUDOU...	PRECISA REINICIAR?	O QUE FAZER
Arquivo <code>.tsx</code> ou <code>.css</code> do frontend	NÃO	Next.js Fast Refresh recarrega o browser sozinho. Só salvar.
Arquivo <code>.ts</code> do backend	NÃO	<code>start:dev</code> reinicia sozinho. Espere ver "Nest application successfully started" no log.
<code>package.json</code> (nova lib)	SIM	<code>Ctrl+C</code> no terminal certo → <code>npm install</code> → sobe de novo.
<code>schema.prisma</code>	SIM	<code>Ctrl+C</code> no backend → <code>npx prisma migrate dev --name "descricao"</code> → <code>npm run start:dev</code> . O migrate dev já regenera o client.
<code>backend\.env</code>	SIM	<code>Ctrl+C</code> → <code>npm run start:dev</code> .
<code>frontend\.env.local</code>	SIM	<code>Ctrl+C</code> → <code>npm run dev</code> .

4. Comandos que você vai usar direto

Docker

```
docker ps                # o que está rodando
docker compose up -d      # sobe Postgres + Redis
docker compose logs -f postgres # ver logs do banco
docker compose down        # para (mantém dados)
docker compose down -v     # para e ZERA o banco
```

Prisma (dentro de `backend/`)

```
npx prisma generate      # regenera o client TypeScript
npx prisma migrate dev --name nome # cria e aplica migration nova
npx prisma migrate deploy # só aplica migrations existentes
npx prisma db seed        # popula dados demo
npx prisma studio         # UI visual em http://localhost:5555
```

Ver e editar dados do banco

```
npx prisma studio # RECOMENDADO – interface web bonita
docker exec -it barbearia_luck_db psql -U postgres barbearia_luck # SQL direto
```

Git

```
git pull origin claude/barbearia-luck-app-LzpRD # baixa atualizações
git status # ver mudanças
git add .
git commit -m "descrição curta"
git push # envia
```

5. Trabalhando com o banco (dev)

Zerar tudo e recomeçar

```
docker compose down -v # apaga o volume
docker compose up -d
cd backend
npx prisma migrate deploy
npx prisma db seed
```

Só refazer o seed (mantém o schema)

```
npx prisma migrate reset --force # zera dados, aplica migrations, roda seed
```

Prisma Studio é ótimo pra testar cenários: você abre o app, cria um agendamento, entra no Studio e vê o registro; edita status pra COMPLETED direto pela UI, volta no app e vê o efeito (ex: pontos de fidelidade subindo).

6. Estrutura do projeto

```
LuckApp/
├── backend/                                # NestJS API (porta 3001)
│   ├── prisma/
│   │   ├── schema.prisma                 # definição das tabelas
│   │   ├── migrations/                   # SQL versionado
│   │   └── seed.ts                       # dados demo
│   └── src/
│       ├── modules/
│       │   ├── auth/                    # JWT + OTP
│       │   ├── owners/                  # donos
│       │   ├── clients/                 # clientes
│       │   ├── units/                   # unidades (filiais)
│       │   ├── barbers/                 # barbeiros
│       │   ├── services/                # serviços do dono
│       │   ├── appointments/            # agendamentos + slots
│       │   ├── availability/            # bloqueios de agenda
│       │   ├── reschedule/              # pedidos pendentes
│       │   ├── loyalty/                 # fidelidade
│       │   ├── financial/               # resumo, relatório, CSV
│       │   └── integrations/            # google-calendar + whatsapp + email
│       └── frontend/                   # Next.js SPA (porta 3000)
│           ├── app/
│           │   ├── page.tsx              # state machine – todas as telas
│           │   ├── components/
│           │   │   ├── luck/             # design system (Header, CTA, Field...)
│           │   │   ├── icons/            # biblioteca de ícones
│           │   │   └── screens/           # 24 telas do app
│           │   └── lib/
│           │       └── services/          # wrappers tipados por domínio da API
│           └── docker-compose.yml         # Postgres + Redis
├── SIMULAR.md                            # passo a passo alternativo
```

7. Contas e dados demo (após `db seed`)

ITEM	VALOR
E-mail do dono	<code>demo@barbearialuck.com</code>
Senha	<code>123456</code>
Unidades	Atlântica (RJ) · Bom Pastor (SP)
Barbeiros	Diego Monteiro · Thiago Alves · Otávio Reis
Serviços	Corte Masculino · Barba · Combo Premium · Sobrancelha

Mocks: WhatsApp e e-mail não enviam de verdade. Os códigos OTP aparecem no **log do backend** (terminal onde roda `npm run start:dev`). Procure por `[WhatsApp]` ou `[E-mail]`.

8. Endpoints principais da API

Autenticação

MÉTODO	ENDPOINT	DESCRIÇÃO
POST	<code>/auth/owner/register</code>	Cadastro do dono
POST	<code>/auth/owner/login</code>	Login e-mail + senha
POST	<code>/auth/owner/send-otp</code>	Login por WhatsApp
POST	<code>/auth/client/send-otp</code>	OTP cliente (WhatsApp)
POST	<code>/auth/client/send-email-otp</code>	OTP cliente (e-mail)
GET	<code>/auth/google/url</code>	URL OAuth Google Calendar

Agendamentos

MÉTODO	ENDPOINT	DESCRIÇÃO
GET	<code>/appointments/available-slots</code>	Slots livres (ownerId+date+serviceld)
POST	<code>/appointments</code>	Cria agendamento
GET	<code>/appointments/owner</code>	Agenda do dono
GET	<code>/appointments/client</code>	Histórico do cliente
PATCH	<code>/appointments/:id/status</code>	Atualiza status
PATCH	<code>/appointments/:id/payment</code>	Registra pagamento

Extensões

MÉTODO	ENDPOINT	DESCRIÇÃO
POST	<code>/availability/blocks/day</code>	Bloquear dia inteiro
POST	<code>/availability/blocks/slot</code>	Bloquear slot específico
POST	<code>/reschedule-requests</code>	Cliente solicita remarcação
PATCH	<code>/reschedule-requests/:id</code>	Dono aprova/recusa
GET	<code>/loyalty/me</code>	Pontos e recompensa
GET	<code>/financial/summary?period=week</code>	Resumo por período
GET	<code>/financial/export.csv</code>	Baixa CSV do relatório

Todos os endpoints com token exigem `Authorization: Bearer <token>`. No frontend, o token fica em `localStorage.token`.

9. Design system (frontend)

Tokens de cor (em `frontend/app/globals.css`)

TOKEN	HEX	USO
<code>--red</code>	#C0392B	CTAs, bordas de destaque, cor do cliente
<code>--red-soft</code>	#C0392B14	Fundos suaves de estado ativo
<code>--navy</code>	#1A3A6B	CTAs do dono, elementos secundários
<code>--navy-soft</code>	#1A3A6B14	Fundos suaves do dono
<code>--bg</code> / <code>--bg2</code>	#FFFFFF / #F5F5F5	Fundo principal / secundário
<code>--gray</code> / <code>--gray-soft</code>	#BDBDBD / #E8E8E8	Divisores / borders
<code>--ink</code>	#2C2C2C	Texto principal
<code>--whatsapp</code>	#25D366	CTA WhatsApp

Tipografia

- **Playfair Display** — títulos serifados
- **Inter** — corpo do texto
- **Bebas Neue** — "eyebrow" (labels em caixa alta)
- **JetBrains Mono** — códigos, horários, valores
- **Dancing Script** — assinatura "Luck"

Componentes reutilizáveis (em `components/Luck/`)

- `LuckHeader` — cabeçalho com logo
- `LuckProgress` — barra de progresso do wizard
- `LuckCTA` — botão principal (variantes: primary/navy/whatsapp/outline/ghost)
- `LuckFooter` — container do CTA no rodapé
- `LuckField` — input com estados idle/focus/filled/success/error
- `LuckSocialButtons` — Google + Facebook (mock)

10. Troubleshooting rápido

SINTOMA	CAUSA	SOLUÇÃO
<code>ECONNREFUSED 5432</code>	Postgres não subiu	<code>docker compose logs postgres</code>
<code>Environment variable not found: DATABASE_URL</code>	Falta o <code>.env</code>	<code>Copy-Item backend\.env.example backend\.env</code>
Docker: <code>Virtualization support not detected</code>	VT-x/AMD-V desabilitado	Habilitar na BIOS + instalar WSL 2 (<code>wsl --install</code>)
<code>bash: comando não reconhecido</code>	Rodou <code>.sh</code> no PowerShell	Use Git Bash ou rode os comandos manualmente
<code>&&</code> não funciona	PowerShell 5 (antigo)	Rode comandos em linhas separadas
Frontend não atualiza	Cache do browser	<code>Ctrl+Shift+R</code> (hard refresh)
Backend em loop de reinício	Erro de import	Apague <code>dist/</code> , corrija o import, reinicie
OTP nunca "chega"	Não olhou o log	Código está no terminal do backend, prefixo <code>[WhatsApp]</code>
Login OTP dono falha	Owner não existe	Use <code>demo@barbearialuck.com</code> + senha
Botão Google Calendar não conecta	Sem credenciais Google	Deixe pra depois (integração externa opcional)
Port 3001 já em uso	Backend rodando em outro terminal	<code>Get-Process node Stop-Process</code>

11. Fluxos completos para testar

A. Cliente novo agenda um horário

1. Abre `/` → **Sou cliente**
2. Escolhe unidade (ex: Atlântica)
3. Informa nome + WhatsApp → **Enviar código**
4. Vai no terminal do backend → copia o OTP do log `[WhatsApp]`
5. Digita o código → escolhe serviço → escolhe barbeiro → escolhe slot → confirma

B. Dono confere e conclui atendimento

1. Logout → Entrar → toggle FUNCIONÁRIO → e-mail + senha
2. Vê o agendamento novo no dashboard
3. Vai no Prisma Studio (`npx prisma studio`)
4. Muda status do Appointment para `COMPLETED`, paymentStatus para `PAID`
5. Volta na conta do cliente → pontos de fidelidade subiram (`GET /loyalty/me`)

C. Reagendamento pendente

1. Como cliente → home → ícone calendário → escolhe novo horário → SOLICITAR
2. Log do backend mostra mensagem WhatsApp pro dono

3. Como dono → dashboard → badge com "1" no header → aprova ou recusa

12. Recursos externos

- **Documentação Prisma:** <https://www.prisma.io/docs>
- **Documentação NestJS:** <https://docs.nestjs.com>
- **Documentação Next.js:** <https://nextjs.org/docs>
- **Docker Desktop:** <https://docs.docker.com/desktop>
- **Repositório:** <https://github.com/Rodrigotxs/LuckApp>
- **Branch atual:** [claude/barbearia-luck-app-LzpRD](#)

Barbearia Luck · Guia de Desenvolvimento · Gerado a partir do repositório [claude/barbearia-luck-app-LzpRD](#)